Natural Language Processing

Fundamentals Glossary

A Complete Reference: Definition • Explanation • Example • Advantages • Disadvantages

1. Core Linguistic Concepts

Tokenization

📌 **Definition:** The process of splitting raw text into smaller units called tokens, which can be words, subwords, characters, or sentences.

💡 **Explanation:** Tokenization is often the very first step in any NLP pipeline. Before a machine can understand language, it needs to break the continuous stream of characters into meaningful chunks. Depending on the task, tokens may be individual words, punctuation marks, or even sub-word units (as in Byte-Pair Encoding).

📝 **Example:** Input: "The cat sat on the mat." → Tokens: ["The", "cat", "sat", "on", "the", "mat", "."]

✅ **Advantages:** Simple and fast to compute; creates a structured input for downstream models; language-agnostic at the character level.

⚠️ **Disadvantages:** Naïve whitespace tokenization fails for languages like Chinese or Japanese; struggles with contractions ("don't") and hyphenated words.

Stemming

📌 **Definition:** A rule-based process that reduces a word to its root/base form by stripping suffixes and prefixes.

💡 **Explanation:** Stemming uses heuristic rules to chop off word endings. For example, the Porter Stemmer removes '-ing', '-ed', '-s' etc. The resulting stem may not be a real word—it is just a normalized form used to group related words together.

📝 **Example:** "running" → "run", "happiness" → "happi", "studies" → "studi"

✅ **Advantages:** Very fast and computationally cheap; reduces vocabulary size; useful for information retrieval tasks.

⚠️ **Disadvantages:** Produces non-real words (over-stemming); can conflate unrelated words (under-stemming); not linguistically accurate.

Lemmatization

📌 **Definition:** The process of reducing a word to its canonical dictionary form (lemma) using vocabulary and morphological analysis.

💡 **Explanation:** Unlike stemming, lemmatization returns real words. It considers the context and part-of-speech to determine the correct base form. For example, 'better' is lemmatized to 'good', while 'running' (verb) becomes 'run' and 'running' (noun/gerund) stays 'running'.

📝 **Example:** "better" → "good", "ran" → "run", "geese" → "goose"

✅ **Advantages:** Produces linguistically valid words; context-aware; more accurate than stemming for complex morphology.

⚠️ **Disadvantages:** Slower than stemming due to dictionary lookups; requires POS tagging for best accuracy; not always available for low-resource languages.

Part-of-Speech (POS) Tagging

📌 **Definition:** The process of assigning grammatical categories (noun, verb, adjective, adverb, etc.) to each token in a text.

💡 **Explanation:** POS tagging is a fundamental disambiguation task. The word 'bank' is a noun in 'river bank' but a verb in 'bank a check'. Taggers use statistical models (Hidden Markov Models, CRFs, or Transformers) trained on annotated corpora like the Penn Treebank.

📝 **Example:** "The quick brown fox jumps" → [The/DT, quick/JJ, brown/JJ, fox/NN, jumps/VBZ]

✅ **Advantages:** Enables downstream tasks like parsing and NER; helps with word sense disambiguation; improves lemmatization accuracy.

⚠️ **Disadvantages:** Context-dependent and error-prone for ambiguous words; performance degrades on informal text (tweets, SMS); requires labeled training data.

Parsing (Syntactic Parsing)

📌 **Definition:** The process of analyzing a sentence's grammatical structure according to formal grammar rules, producing a parse tree.

💡 **Explanation:** Parsing comes in two main varieties: Constituency Parsing (dividing a sentence into nested phrases like NP, VP) and Dependency Parsing (establishing directed relationships between words, such as subject-verb). Modern parsers use neural networks for high accuracy.

📝 **Example:** "She loves NLP." → (S (NP She) (VP loves (NP NLP)))

✅ **Advantages:** Reveals syntactic relationships useful for question answering and relation extraction; enables grammar checking; foundational for semantic understanding.

⚠️ **Disadvantages:** Computationally expensive for long sentences; ambiguous sentences have multiple valid parse trees; degrades on ungrammatical text.

Named Entity Recognition (NER)

📌 **Definition:** The task of identifying and classifying named entities (persons, organizations, locations, dates, etc.) in unstructured text.

💡 **Explanation:** NER is a sequence labeling task where each token receives a tag: B-PER (beginning of person), I-ORG (inside an organization), O (not an entity), etc. Modern NER systems use BiLSTM-CRF or Transformer models fine-tuned on labeled datasets like CoNLL-2003.

📝 **Example:** "Apple was founded by Steve Jobs in Cupertino." → [Apple: ORG, Steve Jobs: PER, Cupertino: LOC]

✅ **Advantages:** Automatically extracts structured facts from free text; widely applicable in finance, healthcare, and journalism; high accuracy with modern models.

⚠️ **Disadvantages:** Struggles with rare or novel entities; entity boundaries are ambiguous; cross-lingual NER remains challenging.

2. Text Preprocessing

Stop Word Removal

📌 **Definition:** The elimination of highly frequent, low-information words (e.g., 'the', 'is', 'at') from text before processing.

💡 **Explanation:** Stop words carry little semantic content and can dominate term frequency statistics. Removing them reduces noise and computational cost. Standard stop word lists exist for most languages (NLTK, spaCy), and custom lists can be built for domain-specific needs.

📝 **Example:** "The cat is on the mat" → "cat mat" (after removing stop words)

✅ **Advantages:** Reduces data dimensionality; speeds up processing; can improve model performance on bag-of-words tasks.

⚠️ **Disadvantages:** Removing 'not' or 'no' can reverse sentence meaning; harmful for tasks like sentiment analysis or question answering; stop word lists are language/domain-specific.

Text Normalization

📌 **Definition:** The process of transforming text into a standard, consistent format by handling casing, punctuation, contractions, numbers, and special characters.

💡 **Explanation:** Raw text is noisy—it contains uppercase letters, emojis, HTML tags, URLs, and abbreviations. Normalization ensures the model sees clean, uniform input. Steps can include lowercasing, expanding contractions ('won't' → 'will not'), removing HTML, and standardizing unicode.

📝 **Example:** "I CAN'T believe it's 2024!!!" → "i cannot believe it is 2024"

✅ **Advantages:** Reduces vocabulary size; ensures consistency; improves model generalization across different writing styles.

⚠️ **Disadvantages:** Lowercasing loses information ("US" vs "us"); over-normalization removes useful signals; language-specific rules are needed.

N-grams

📌 **Definition:** Contiguous sequences of N items (words, characters, or tokens) from a given text.

💡 **Explanation:** A unigram (N=1) is a single word, a bigram (N=2) is two consecutive words, a trigram (N=3) is three words, and so on. N-gram models capture local context and are used in language modeling, text classification, and spell checking. Character N-grams are useful for morphologically rich languages.

📝 **Example:** "I love NLP" → Unigrams: [I, love, NLP]; Bigrams: [I love, love NLP]; Trigrams: [I love NLP]

✅ **Advantages:** Simple to compute; captures local word context; character N-grams handle unseen words and misspellings.

⚠️ **Disadvantages:** Data sparsity grows exponentially with N; fails to capture long-range dependencies; large N-gram models require huge storage.

Bag of Words (BoW)

📌 **Definition:** A text representation that encodes a document as a vector of word frequencies or presence indicators, ignoring word order and grammar.

💡 **Explanation:** BoW creates a vocabulary of all unique words across a corpus. Each document is then represented as a fixed-length vector where each dimension corresponds to a word and the value is its count (or a binary 1/0). Despite its simplicity, BoW is effective for many classification tasks.

📝 **Example:** Vocab: [cat, dog, sat, mat] — "cat sat" → [1,0,1,0]; "dog sat on mat" → [0,1,1,1]

✅ **Advantages:** Simple to implement; interpretable; works well with sparse linear models (Naive Bayes, SVM).

⚠️ **Disadvantages:** Ignores word order ('dog bites man' vs 'man bites dog'); high-dimensional and sparse; doesn't capture semantics or context.

TF-IDF (Term Frequency–Inverse Document Frequency)

📌 **Definition:** A numerical statistic that reflects the importance of a word to a document in a corpus by balancing its local frequency against its global rarity.

💡 **Explanation:** TF (Term Frequency) measures how often a word appears in a document. IDF (Inverse Document Frequency) penalizes words that appear in many documents (like 'the'). The product TF×IDF gives a high score to words that are frequent in a specific document but rare across the corpus—these are the most characteristic words.

📝 **Example:** The word 'neural' appearing 10 times in one NLP paper and rarely in other documents gets a high TF-IDF score for that paper.

✅ **Advantages:** Handles common word downweighting automatically; simple and widely used for information retrieval and document similarity.

⚠️ **Disadvantages:** Still ignores word order and semantics; doesn't generalize across synonyms; less effective than dense embeddings for deep semantic tasks.

---

3. Word Representations & Embeddings

One-Hot Encoding

📌 **Definition:** A vector representation where each word is encoded as a binary vector with a single '1' at its index position and '0' everywhere else.

💡 **Explanation:** If the vocabulary has 10,000 words, each word is a vector of length 10,000 with exactly one non-zero entry. This is a simple but naive approach: 'cat' and 'kitten' are as far apart as 'cat' and 'democracy', because the vectors are orthogonal (dot product = 0).

📝 **Example:** Vocab [cat, dog, bird]: cat=[1,0,0], dog=[0,1,0], bird=[0,0,1]

✅ **Advantages:** Trivial to construct; no training required; completely unambiguous representation.

⚠️ **Disadvantages:** Extremely high-dimensional; no semantic similarity information; doesn't scale with large vocabularies.

Word Embeddings

📌 **Definition:** Dense, low-dimensional continuous vector representations of words that capture semantic and syntactic relationships learned from large corpora.

💡 **Explanation:** Instead of a 10,000-dimensional sparse vector, a word is represented as a 100–300 dimensional dense vector. Words with similar meanings cluster together in this embedding space. The famous example: vector('King') − vector('Man') + vector('Woman') ≈ vector('Queen').

📝 **Example:** "king" → [0.25, -0.71, 0.14, ...] (300-dim vector); similar to "queen", "monarch"

✅ **Advantages:** Captures semantic relationships; dimensionality reduction; transferable across tasks; enables analogy reasoning.

⚠️ **Disadvantages:** Static embeddings assign one vector per word regardless of context ('bank' always has the same vector); requires large training corpora; vectors are hard to interpret.

Word2Vec

📌 **Definition:** A shallow neural network model that learns word embeddings using two architectures: Continuous Bag of Words (CBOW) and Skip-gram.

💡 **Explanation:** CBOW predicts a target word from its surrounding context words. Skip-gram does the reverse: predicts context words given a target word. Both are trained with large text

corpora using stochastic gradient descent. The resulting embeddings encode distributional semantics ('a word is known by the company it keeps').

📝 **Example:** Skip-gram: given 'NLP', predict ['I', 'love', 'a', 'lot']; CBOW: given context, predict 'NLP'

✅ **Advantages:** Efficient to train on large corpora; produces high-quality semantic embeddings; widely available pre-trained models.

⚠️ **Disadvantages:** Fixed context window; polysemous words get a single vector; no subword information (out-of-vocabulary words fail).

GloVe (Global Vectors for Word Representation)

📌 **Definition:** A word embedding method that learns vectors by factorizing the global word co-occurrence matrix of a corpus.

💡 **Explanation:** GloVe constructs a co-occurrence matrix counting how often pairs of words appear together within a window. It then trains embeddings such that the dot product of two word vectors approximates the log of their co-occurrence probability. This combines local context (like Word2Vec) with global statistical information.

📝 **Example:** Co-occurrence of 'ice' and 'solid' vs 'ice' and 'gas' is captured in the ratio of their probabilities, encoded in the embedding geometry.

✅ **Advantages:** Leverages global statistics; often outperforms Word2Vec on word analogy tasks; efficient training via matrix factorization.

⚠️ **Disadvantages:** Still context-independent (one vector per word); requires entire corpus in memory to build co-occurrence matrix; same limitations as static embeddings.

Contextual Embeddings (ELMo, BERT)

📌 **Definition:** Dynamic word representations where the vector for each word is computed based on its surrounding context, allowing different senses of a word to have different vectors.

💡 **Explanation:** ELMo (Embeddings from Language Models) generates contextual embeddings from a bidirectional LSTM. BERT uses a Transformer encoder to produce deep contextual representations by pre-training on masked language modeling. The vector for 'bank' in 'river bank' differs from 'bank account'.

📝 **Example:** "He went to the bank." (river context) vs. "He deposited money at the bank." (financial context) → different vectors for 'bank'

✅ **Advantages:** Handles polysemy naturally; captures long-range context; state-of-the-art performance on almost all NLP tasks.

⚠️ **Disadvantages:** Computationally expensive; large model size; requires GPU for efficient inference; harder to interpret than static embeddings.

Subword Tokenization (BPE, WordPiece)

📌 **Definition:** A tokenization strategy that splits words into sub-word units to handle out-of-vocabulary words and morphologically rich languages.

💡 **Explanation:** Byte-Pair Encoding (BPE) starts with characters and iteratively merges the most frequent adjacent pairs until a target vocabulary size is reached. WordPiece (used by BERT) uses a likelihood-based criterion. This lets models represent unseen words like 'unfriendliness' as ['un', '##friend', '##li', '##ness'].

📝 **Example:** "unbelievable" → ["un", "##believ", "##able"] (WordPiece notation)

✅ **Advantages:** Handles rare and unseen words; reduces vocabulary size compared to word-level; language-agnostic; captures morphology.

⚠️ **Disadvantages:** Tokens don't align with human-intuitive word boundaries; longer token sequences increase computational cost; requires training the tokenizer separately.

---

4. Language Models

N-gram Language Model

📌 **Definition:** A probabilistic model that estimates the probability of a word given the previous N−1 words using counts from a training corpus.

💡 **Explanation:** A bigram model estimates P(word | previous_word). A trigram model uses two preceding words. Probabilities are estimated by counting occurrences and applying smoothing techniques (Laplace, Kneser-Ney) to handle unseen N-grams. These were dominant before neural approaches.

📝 **Example:** P("the" | "cat sat on") from a trigram model is estimated from the count of 'cat sat on the' divided by 'cat sat on' in the corpus.

✅ **Advantages:** Simple, fast, and interpretable; no GPU required; useful baselines for speech recognition and machine translation.

⚠️ **Disadvantages:** Suffers from data sparsity; limited context window (N words); cannot generalize to unseen word combinations.

Perplexity

📌 **Definition:** A metric for evaluating language models that measures how well the model predicts a held-out test corpus; lower is better.

💡 **Explanation:** Perplexity is the exponentiated average negative log-likelihood per token: $PPL = \exp(-(1/N)\ \Sigma \log P(w_i))$. Intuitively, it measures how 'surprised' a model is by new text. A model with perplexity 100 is as confused as if it had to choose uniformly among 100 words at each step.

📝 **Example:** A trigram model might have perplexity 200 on news text; a Transformer LM might achieve perplexity 20 on the same data.

✅ **Advantages:** Widely used, comparable across models on the same dataset; directly tied to model log-likelihood; fast to compute.

⚠️ **Disadvantages:** Dataset-specific (perplexity on different corpora is not comparable); doesn't directly measure downstream task performance; doesn't penalize hallucination.

Recurrent Neural Network (RNN)

📌 **Definition:** A neural network architecture with recurrent connections that maintain a hidden state, enabling sequential processing of variable-length inputs.

💡 **Explanation:** At each time step, an RNN takes the current input token and the previous hidden state to produce a new hidden state. This allows information to flow across the sequence. However, vanilla RNNs suffer from vanishing gradients, making it hard to learn long-range dependencies.

📝 **Example:** Processing "I love NLP" step by step: h1=f(h0,'I'), h2=f(h1,'love'), h3=f(h2,'NLP')—h3 encodes the whole sentence.

✅ **Advantages:** Handles variable-length sequences naturally; parameter sharing across time steps; foundational architecture for sequence modeling.

⚠️ **Disadvantages:** Vanishing and exploding gradients; slow sequential computation (not parallelizable); limited long-range memory without gating mechanisms.

LSTM (Long Short-Term Memory)

📌 **Definition:** A type of RNN with gating mechanisms (input, forget, output gates) that selectively retain or discard information, solving the vanishing gradient problem.

💡 **Explanation:** LSTMs maintain a cell state (long-term memory) and a hidden state (short-term memory). Three gates control information flow: the forget gate decides what to erase, the input gate controls what new information to write, and the output gate decides what to output. This enables learning dependencies across hundreds of time steps.

📝 **Example:** In machine translation, an LSTM encoder summarizes an entire source sentence into a context vector; the decoder generates the translation word by word.

✅ **Advantages:** Handles long-range dependencies much better than vanilla RNNs; effective for language modeling, translation, and speech recognition.

⚠️ **Disadvantages:** Still sequential (not parallelizable); slower to train than Transformers; largely superseded by Transformer-based models on most NLP benchmarks.

Transformer Architecture

📌 **Definition:** A neural network architecture based entirely on self-attention mechanisms (without recurrence or convolution) that processes sequences in parallel.

💡 **Explanation:** Introduced in 'Attention Is All You Need' (Vaswani et al., 2017), the Transformer uses multi-head self-attention to relate every token to every other token simultaneously. This allows parallel computation and capture of long-range dependencies. It consists of stacked encoder and/or decoder layers with attention, feed-forward sub-layers, residual connections, and layer normalization.

📝 **Example:** BERT (encoder-only), GPT (decoder-only), and T5 (encoder-decoder) are all Transformer variants that have set state-of-the-art results on dozens of NLP benchmarks.

✅ **Advantages:** Fully parallelizable training; captures global context from the first layer; scales remarkably well with more data and parameters.

⚠️ **Disadvantages:** Self-attention is $O(n^2)$ in sequence length (quadratic cost); requires large datasets and compute; prone to hallucination at scale.

Attention Mechanism

📌 **Definition:** A mechanism that allows a model to dynamically weight and focus on different parts of the input when producing each element of the output.

💡 **Explanation:** For each output position, attention computes a weighted sum over all input positions. Weights (attention scores) are computed via dot products of Query (Q) and Key (K) vectors, scaled and softmaxed, then used to weight Value (V) vectors. Multi-head attention runs this process H times in parallel with different learned projections.

📝 **Example:** Translating 'The animal didn't cross the street because it was too tired' — the model attends strongly to 'animal' when generating the translation of 'it'.

✅ **Advantages:** Enables interpretability through attention maps; captures arbitrary-length dependencies; key to the Transformer's success.

⚠️ **Disadvantages:** Quadratic memory and computation with sequence length; attention maps don't always reflect actual reasoning; can be noisy in practice.

5. Pre-training & Transfer Learning

Pre-training

📌 **Definition:** Training a model on a large general-purpose dataset using self-supervised objectives before adapting it to a specific downstream task.

💡 **Explanation:** Pre-training leverages massive unlabeled text corpora (web pages, books, Wikipedia). Common objectives include Masked Language Modeling (BERT), Causal Language Modeling (GPT), and Span Corruption (T5). The model learns rich linguistic representations that transfer to many tasks with minimal additional training.

📝 **Example:** BERT is pre-trained on 3.3 billion words from BookCorpus and Wikipedia, then fine-tuned on question answering datasets like SQuAD.

✅ **Advantages:** Dramatically reduces labeled data requirements for downstream tasks; leads to state-of-the-art results; reusable across many tasks.

⚠️ **Disadvantages:** Extremely compute and data intensive; environmental cost; pre-training biases can propagate to downstream applications.

Fine-tuning

📌 **Definition:** The process of continuing to train a pre-trained model on a smaller, task-specific labeled dataset to adapt it to a particular application.

💡 **Explanation:** During fine-tuning, the pre-trained weights serve as initialization. All (or some) parameters are updated via gradient descent on the labeled downstream data. For BERT, a task-specific classification head is added on top of the [CLS] token representation. Fine-tuning typically requires far fewer examples than training from scratch.

📝 **Example:** Fine-tuning BERT on the IMDB sentiment dataset (25K examples) achieves >94% accuracy in a few hours, vs. weeks of pre-training.

✅ **Advantages:** Achieves high performance with limited labeled data; fast to converge; leverages pre-learned linguistic knowledge.

⚠️ **Disadvantages:** Risk of catastrophic forgetting of pre-trained knowledge; sensitive to hyperparameters (learning rate); can overfit on small datasets.

Prompt Engineering

📌 **Definition:** The practice of designing and optimizing input prompts (text instructions) to elicit desired behaviors from large language models without changing model weights.

💡 **Explanation:** Instead of fine-tuning, prompts guide the model's generation. Techniques include zero-shot prompting (just describing the task), few-shot prompting (providing input-output examples in the context), chain-of-thought prompting (asking the model to reason step by step), and system prompts (setting the model's persona).

📝 **Example:** Zero-shot: 'Translate to French: Hello World.' | Few-shot: 'English: cat → French: chat; English: dog → French:' → 'chien'

✅ **Advantages:** No gradient updates or labeled data required; flexible and fast to iterate; enables adaptation to novel tasks in real time.

⚠️ **Disadvantages:** Sensitive to exact wording; inconsistent across runs; not as reliable as fine-tuning for complex or specialized tasks; prompt injection risks.

Transfer Learning

📌 **Definition:** A machine learning paradigm where knowledge learned from one task or domain is leveraged to improve performance on a different but related task or domain.

💡 **Explanation:** In NLP, transfer learning typically means using a pre-trained language model as the starting point and adapting it to a new task. The underlying idea is that linguistic features (syntax, morphology, world knowledge) learned on large corpora transfer to specialized tasks like medical NER or legal document classification.

📝 **Example:** A model pre-trained on general English text can be fine-tuned on clinical notes to achieve strong performance on medical entity extraction.

✅ **Advantages:** Reduces need for large labeled datasets; faster convergence; enables high performance in low-resource settings.

⚠️ **Disadvantages:** Domain mismatch can limit transfer effectiveness; large pre-trained models are costly to store and deploy; may inherit biases from pre-training data.

---

6. Core NLP Tasks

Sentiment Analysis

📌 **Definition:** The task of computationally identifying and categorizing subjective information (positive, negative, neutral) in text.

💡 **Explanation:** Sentiment analysis ranges from document-level (is this review positive or negative?) to aspect-level (how does the user feel about the camera quality specifically?). It can be approached with lexicon-based methods (SentiWordNet), traditional ML (SVM + TF-IDF), or fine-tuned Transformers.

📝 **Example:** "The battery life is amazing but the camera is disappointing." → Battery: Positive, Camera: Negative (aspect-level)

✅ **Advantages:** High commercial value for brand monitoring, customer feedback analysis, financial markets; well-studied with many benchmarks.

⚠️ **Disadvantages:** Sarcasm and irony are very difficult to detect; cultural context affects sentiment; aspect-level analysis is complex.

Machine Translation (MT)

📌 **Definition:** Automatic translation of text from one natural language to another using computational methods.

💡 **Explanation:** Modern MT is dominated by Neural Machine Translation (NMT) using Transformer encoder-decoder architectures. Models like Google Translate and DeepL are trained on millions of parallel sentence pairs. Beam search is used during decoding to find high-probability translations.

📝 **Example:** "Bonjour le monde" (French) → "Hello the world" (English, word-for-word) | "Hello, world!" (correct MT output)

✅ **Advantages:** Enables global communication; scales to hundreds of language pairs; near-human quality for high-resource language pairs.

⚠️ **Disadvantages:** Still weak for low-resource language pairs; struggles with domain-specific terminology; can hallucinate fluent but incorrect translations.

Text Summarization

📌 **Definition:** The task of automatically producing a shorter version of one or more documents while preserving key information and meaning.

💡 **Explanation:** Extractive summarization selects and concatenates the most important sentences directly from the source text. Abstractive summarization generates novel text that may not appear verbatim in the source—more human-like but harder to train. Both are evaluated using ROUGE scores.

📝 **Example:** Extractive: picks sentences 1, 3, 7 from a news article. Abstractive: generates 'Researchers at MIT developed a new vaccine that is 95% effective.'

✅ **Advantages:** Saves reading time; scalable for large document collections; useful in news, legal, and scientific domains.

⚠️ **Disadvantages:** Abstractive models can hallucinate facts; extractive models can be incoherent; evaluation metrics (ROUGE) don't fully capture quality.

Question Answering (QA)

📌 **Definition:** The task of automatically producing accurate answers to natural language questions, either from a provided context or from stored knowledge.

💡 **Explanation:** Extractive QA (SQuAD) finds the answer span within a given passage. Open-domain QA retrieves relevant passages from a large corpus first (using dense retrieval or BM25), then extracts or generates an answer. Generative QA (as in ChatGPT) generates answers from parametric knowledge.

📝 **Example:** Context: 'Marie Curie won the Nobel Prize in 1903.' Question: 'Who won the Nobel Prize in 1903?' Answer: 'Marie Curie'

✅ **Advantages:** High practical value for search engines, customer support, and knowledge management; well-defined evaluation with benchmark datasets.

⚠️ **Disadvantages:** Multi-hop reasoning (across multiple documents) remains challenging; models can confidently give incorrect answers; adversarial robustness is weak.

Text Classification

📌 **Definition:** The task of assigning predefined categories or labels to text documents based on their content.

💡 **Explanation:** Applications include spam detection (spam/not spam), topic classification (sports, politics, technology), intent detection in chatbots, and language identification. Features can be TF-IDF vectors fed to logistic regression, or raw text fed to fine-tuned Transformers.

📝 **Example:** Email text → [Spam / Not Spam]; News article → [Sports / Politics / Tech / Entertainment]

✅ **Advantages:** Well-understood problem with many available datasets and tools; efficient inference; high accuracy achievable with pre-trained models.

⚠️ **Disadvantages:** Requires labeled training data; class imbalance can skew models; multi-label classification is significantly harder.

Relation Extraction

📌 **Definition:** The task of identifying and classifying semantic relationships between entities mentioned in text.

💡 **Explanation:** Given a sentence with two entities, relation extraction determines the relationship type between them from a predefined schema. For example, 'founded_by', 'located_in', 'works_for'. This is crucial for knowledge graph construction and information extraction pipelines.

📝 **Example:** "Elon Musk founded SpaceX in 2002." → (Elon Musk, founded_by, SpaceX)

✅ **Advantages:** Enables automatic knowledge base population; foundational for question answering and fact verification; reduces manual curation effort.

⚠️ **Disadvantages:** Hard to handle implicit relationships; predefined schemas limit flexibility; cross-sentence relations are very challenging.

Coreference Resolution

📌 **Definition:** The task of identifying all expressions in a text that refer to the same real-world entity.

💡 **Explanation:** When a text uses pronouns and noun phrases to refer to previously introduced entities, coreference resolution links these mentions. For example, 'Obama said he would attend'—'Obama' and 'he' are coreferent. This is essential for coherent text understanding and summarization.

📝 **Example:** "Maria went to the store. She bought apples." → Coreference chain: {Maria, She}

✅ **Advantages:** Enables correct pronoun resolution for downstream tasks; critical for coherent summarization and information extraction.

⚠️ **Disadvantages:** Ambiguous pronoun references require world knowledge; scales poorly with document length; costly to annotate training data.

---

7. Evaluation Metrics

Precision, Recall & F1 Score

📌 **Definition:** Metrics for evaluating classification models: Precision = correct positives / all predicted positives; Recall = correct positives / all actual positives; F1 = harmonic mean of both.

💡 **Explanation:** In NER, Precision asks: of all entities the model predicted, how many are actually correct? Recall asks: of all real entities, how many did the model find? F1 balances the two. Macro-F1 averages across all classes; Micro-F1 computes globally across all instances.

📝 **Example:** Model predicts 10 entities, 8 are correct (Precision=0.8). There are 12 real entities (Recall=8/12≈0.67). F1 = 2×(0.8×0.67)/(0.8+0.67) ≈ 0.73

✅ **Advantages:** Handles class imbalance better than accuracy; decomposes into complementary aspects of model quality; widely accepted across NLP tasks.

⚠️ **Disadvantages:** Choosing β in Fβ score requires domain expertise; macro vs micro distinction is often confused; doesn't measure ranking quality.

BLEU Score

📌 **Definition:** Bilingual Evaluation Understudy (BLEU) is an automated metric for evaluating machine translation quality by comparing N-gram overlap between model output and reference translations.

💡 **Explanation:** BLEU computes a modified N-gram precision (for N=1,2,3,4) between the candidate translation and one or more reference translations, then applies a brevity penalty to discourage overly short translations. Scores range from 0 to 1 (higher is better). BLEU of 0.6+ is considered near human quality.

📝 **Example:** Reference: 'The cat sat on the mat.' Candidate: 'The cat is on the mat.' — BLEU captures 1-gram and 2-gram overlaps.

✅ **Advantages:** Fast, automatic, and reproducible; strongly correlated with human judgments at the corpus level; language-agnostic.

⚠️ **Disadvantages:** Doesn't capture semantics (synonym-unaware); heavily penalizes valid paraphrases; poor correlation with human judgments at the sentence level.

ROUGE Score

📌 **Definition:** Recall-Oriented Understudy for Gisting Evaluation — a set of metrics measuring overlap between generated summaries and reference summaries.

💡 **Explanation:** ROUGE-N measures N-gram overlap (ROUGE-1 for unigrams, ROUGE-2 for bigrams). ROUGE-L measures the longest common subsequence. ROUGE-SU considers skip-bigrams. ROUGE scores report precision, recall, and F1. They are the standard evaluation metric for summarization tasks.

📝 **Example:** System summary: 'The president visited France.' Reference: 'The president traveled to France.' ROUGE-1 F1 = overlapping unigrams / total.

✅ **Advantages:** Simple to compute; widely accepted for summarization; multiple variants capture different aspects of overlap.

⚠️ **Disadvantages:** Content word overlap doesn't guarantee factual accuracy; doesn't capture readability or coherence; penalizes abstractive paraphrasing.

BERTScore

📌 **Definition:** A reference-based evaluation metric that computes semantic similarity between generated and reference texts using contextual BERT embeddings.

💡 **Explanation:** Instead of exact N-gram matching, BERTScore computes token-level cosine similarity between BERT embeddings of the generated text and the reference. It then reports precision, recall, and F1 over the best-matching token pairs. This captures semantic equivalence that BLEU and ROUGE miss.

📝 **Example:** 'The automobile crashed' scores high against 'The car collided' with BERTScore, but low with BLEU (no exact word overlap).

✅ **Advantages:** Captures semantic similarity; handles synonyms and paraphrases; better correlation with human judgments than BLEU.

⚠️ **Disadvantages:** Requires running a large language model for evaluation (computationally expensive); less interpretable than N-gram metrics; scores vary by BERT model version.

---

8. Advanced Concepts

Large Language Models (LLMs)

📌 **Definition:** Transformer-based language models with billions of parameters trained on internet-scale text corpora that exhibit emergent capabilities across diverse tasks.

💡 **Explanation:** LLMs like GPT-4, Claude, and LLaMA are trained on trillions of tokens using causal language modeling (predict the next token). They exhibit emergent abilities—capabilities not present in smaller models—such as few-shot learning, instruction following, code generation, and chain-of-thought reasoning.

📝 **Example:** GPT-4 can write essays, solve math problems, generate code, translate languages, and engage in open-ended conversation—all without task-specific fine-tuning.

✅ **Advantages:** Extremely versatile; strong few-shot learners; enable rapid prototyping of NLP applications; continually improving with scale.

⚠️ **Disadvantages:** Hallucination of plausible but false information; high compute and energy cost; black-box reasoning; difficult to control reliably; potential for misuse.

Retrieval-Augmented Generation (RAG)

📌 **Definition:** A technique that combines a retrieval system with a generative model, allowing the model to ground its responses in externally retrieved documents.

💡 **Explanation:** In RAG, a query is first used to retrieve relevant documents from a knowledge base (using dense or sparse retrieval). The retrieved documents are then concatenated with the query and fed to a generative LLM, which produces a response grounded in the retrieved facts. This reduces hallucination and keeps knowledge up to date.

📝 **Example:** User: 'What are the latest FDA drug approvals?' → Retriever fetches recent FDA news pages → LLM generates a response based on those pages.

✅ **Advantages:** Reduces hallucination by grounding generation in retrieved facts; enables up-to-date knowledge without retraining; improves verifiability.

⚠️ **Disadvantages:** Retrieval quality is a bottleneck; long context from many documents increases latency; noisy retrieved documents can mislead the generator.

Named Entity Disambiguation (NED) / Entity Linking

📌 **Definition:** The task of linking entity mentions in text to their corresponding entries in a knowledge base (e.g., Wikipedia or Wikidata).

💡 **Explanation:** NED follows NER. Once an entity is detected, it must be linked to a specific knowledge base entry to resolve ambiguity. 'Paris' could refer to Paris, France or Paris Hilton. Systems use context, entity descriptions, and embedding similarity to select the correct entry.

📝 **Example:** "Apple released a new product." → Apple → Q312 (Apple Inc., Wikidata) [not the fruit]

✅ **Advantages:** Connects unstructured text to structured knowledge; enables fact verification; essential for knowledge graph population.

⚠️ **Disadvantages:** Ambiguity is very difficult; rare or newly emerging entities may not exist in the knowledge base; multilinguality adds complexity.

Word Sense Disambiguation (WSD)

📌 **Definition:** The computational task of determining which sense of a polysemous word is used in a given context.

💡 **Explanation:** Many words have multiple meanings ('bank', 'bat', 'run'). WSD uses contextual information—neighboring words, syntactic structure, topic—to select the correct sense from a predefined inventory (typically WordNet). Knowledge-based approaches use graph proximity in WordNet; supervised approaches train classifiers per word.

📝 **Example:** "She played the bass guitar" → bass = musical instrument sense. "He caught a bass" → bass = fish sense.

✅ **Advantages:** Improves accuracy of downstream tasks like MT and QA; enables grounding of text in structured semantic resources.

⚠️ **Disadvantages:** Sense inventories in dictionaries can be overly fine-grained; difficult to annotate consistently; LLMs often handle WSD implicitly without explicit WSD modules.

Semantic Role Labeling (SRL)

📌 **Definition:** The task of identifying the predicate-argument structure of a sentence by labeling 'who did what to whom, where, when, and how'.

💡 **Explanation:** SRL annotates verbs (predicates) and their arguments with semantic roles such as Agent (doer), Patient (receiver), Instrument, Location, and Time. PropBank and FrameNet are standard annotation schemas. SRL provides a shallow semantic representation bridging syntax and meaning.

📝 **Example:** "The chef [Agent] cooked [predicate] the pasta [Patient] in the kitchen [Location]."

✅ **Advantages:** Provides portable semantic representation across different surface forms; useful for information extraction, QA, and summarization.

⚠️ **Disadvantages:** Computationally expensive; ambiguity in role assignment; performance drops significantly in cross-domain settings.

Text Generation & Decoding Strategies

📌 **Definition:** The process of producing natural language output from a language model, with various algorithms controlling how tokens are selected at each step.

💡 **Explanation:** Greedy decoding always picks the highest-probability next token (fast but repetitive). Beam search maintains multiple hypotheses simultaneously. Sampling introduces randomness—temperature scaling sharpens or flattens the distribution. Top-k sampling draws from the k most likely tokens; Top-p (nucleus) sampling draws from the smallest set whose cumulative probability ≥ p.

📝 **Example:** Greedy: 'The cat sat on the mat the mat the mat...' (repetition loop). Nucleus sampling (p=0.9) produces more diverse and natural text.

✅ **Advantages:** Diverse strategies for different use cases (creative vs. deterministic); nucleus sampling produces near-human fluent text; beam search optimizes log-likelihood.

⚠️ **Disadvantages:** Greedy and beam search can be repetitive; sampling introduces randomness making outputs non-reproducible; exposure bias between training and inference.

— End of NLP Fundamentals Glossary —